

Compilador HULK

Reporte de Arquitectura, Diseño y Extensiones

Proyecto de Compilación

Integrantes

Rachel Mojena González	312	CC
Melissa Maureen Sales Brito	311	CC
Jean Manuel Martínez Pardo	312	CC

URL del proyecto

<https://github.com/Jean1408Man/Hulk-Compiler>

Junio de 2026

Índice

1. Introducción	3
2. Pipeline de compilación	3
2.1. Lexer	4
2.2. Parser	4
2.3. Árbol de Sintaxis Abstracta (AST)	4
2.4. Análisis semántico	5
2.5. Evaluador de árbol (Camino A)	6
2.6. Generación de IR (Camino B)	6
2.7. Lowering a BannerIR	6
2.8. BannerVM	6
2.9. Invocación del compilador	7
3. Features implementados del lenguaje base	7
4. Las representaciones intermedias	8
4.1. Ejemplo de descenso	9
4.2. Por qué tres niveles	9
5. El sistema de tipos en detalle	9
5.1. Kinds de tipo	10
5.2. La relación de conformancia	10
5.3. El Ancestro Común Más Bajo (LCA)	10
6. Extensión a HULK: type holes explícitos	11
6.1. Motivación	11
6.2. Sintaxis	11
6.3. Semántica	12
6.4. Algoritmo de resolución	12
6.5. Diagramas de inferencia paso a paso	13
6.6. Casos de error de la extensión	14
6.7. La bandera <code>--restricted-inference</code>	14
6.8. Cobertura de pruebas de la extensión	15
7. Análisis comparativo con otros lenguajes	15
7.1. El placeholder de tipo a través de los lenguajes	15
7.2. Typed holes: la inspiración de Haskell e Idris	16
7.3. Por qué punto fijo y no Hindley-Milner	16
7.4. Conformancia estructural vs. nominal	17
7.5. Mapas laterales vs. anotación en el nodo	17
7.6. Modelo de ejecución: VM de pila frente a compilación nativa	18
8. Manejo de errores y diagnósticos	18
8.1. Spans y localización	18
8.2. Niveles y severidades	18
8.3. La propagación absorbente de <code>Error</code>	18

9. Estrategia de pruebas	19
9.1. Pruebas de salida esperada por fase	19
9.2. Pruebas diferenciales	19
9.3. Pruebas end-to-end y de la extensión	19
10.Limitaciones notables	19
11.Conclusiones	20

1 Introducción

HULK (*Havana University Language for Kompilers*) es un lenguaje didáctico de tipado estático con inferencia de tipos opcional, orientado a objetos y con características funcionales. El presente reporte documenta la arquitectura del compilador desarrollado, las decisiones de diseño más relevantes, los features del lenguaje implementados conforme a la especificación oficial y, de manera central para la evaluación de la asignatura, la definición y el análisis detallado de la extensión propuesta al lenguaje: los *type holes* explícitos mediante `_` y `auto`, junto con la bandera `--restricted-inference`.

El compilador traduce archivos fuente `.hulk` a través de un pipeline de múltiples etapas que culmina en la ejecución sobre una máquina virtual de pila propia denominada *BannerVM*. Paralelamente existe un evaluador de árbol que sirve como camino de ejecución alternativo, utilizado para pruebas en etapas tempranas del desarrollo y como oráculo de referencia diferencial. Ambos caminos comparten todas las fases anteriores al backend —lexer, parser y análisis semántico— y divergen únicamente en la etapa de ejecución.

El documento se organiza como sigue. La Sección 2 describe el pipeline de compilación fase a fase. La Sección 3 enumera los features del lenguaje base implementados. La Sección 4 detalla las representaciones intermedias con un ejemplo de descenso, y la Sección 5 formaliza el sistema de tipos (kinds, conformancia y LCA). La Sección 6 define formalmente la extensión de *type holes*, analiza su semántica, su algoritmo de resolución, sus trazas de inferencia y sus casos de error. La Sección 7 compara la extensión y las decisiones de diseño del compilador con un conjunto de lenguajes de programación contemporáneos (C++, Rust, C#, Haskell, TypeScript, Go, Scala y Kotlin), justificando las elecciones tomadas. Las Secciones 8 y 9 cubren el manejo de diagnósticos y la estrategia de pruebas. La Sección 10 discute las limitaciones conocidas y la Sección 11 concluye.

2 Pipeline de compilación

El pipeline corre en ocho fases secuenciales. Cada fase consume la salida de la anterior y produce una nueva representación; ninguna fase muta estructuras producidas por etapas anteriores. Esta inmutabilidad es una decisión de diseño deliberada: permite probar cada fase de forma aislada y razonar sobre el flujo de datos sin efectos colaterales ocultos.

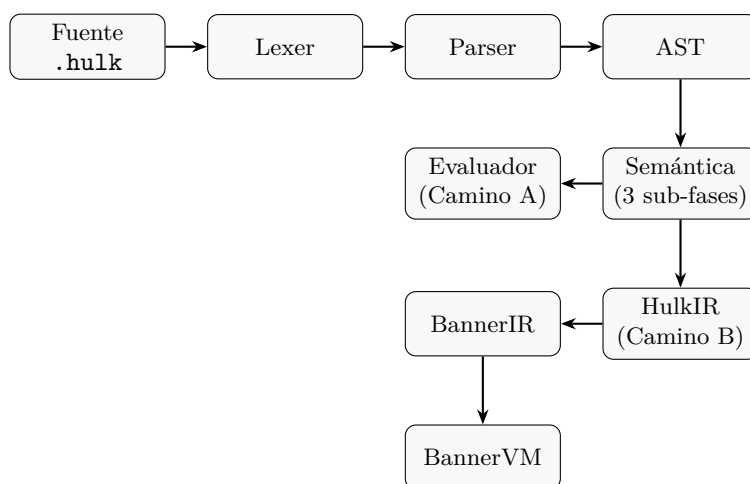


Figura 1: Vista general del pipeline. Tras la semántica el flujo se bifurca en el evaluador de árbol (Camino A) y la generación de IR que desemboca en el BannerVM (Camino B).

2.1 Lexer

El lexer vive en `src/lexer/` y recorre la cadena fuente produciendo una lista plana de *tokens*, cada uno con su *kind*, su lexema original y un *Span* que registra la línea y columna de inicio y fin. Ese span se propaga a través de todas las fases posteriores para que los mensajes de error puedan señalar la ubicación exacta en el código.

El núcleo de reconocimiento es un autómata finito no determinista (AFN). Cada clase de token se describe mediante una expresión regular construida en C++ con combinadores (`lit`, `range`, `seq`, `alt`, `star`, `plus`, `opt`), y de esa lista ordenada de reglas se compila, por la construcción de Thompson, un único AFN que se arma una sola vez y se reutiliza para todo el archivo. El reconocimiento se hace por simulación directa de conjunto de estados con cierre- ϵ , sin determinar a un AFD, aplicando maximal munch y desempataando por orden de declaración; así se reproducen de forma declarativa los casos de longest match (`:=` sobre `:`, `==/>` sobre `=`, `@@` sobre `@`).

La recuperación de errores está integrada: en lugar de lanzar una excepción ante un carácter inválido, el lexer registra un diagnóstico en el `DiagnosticEngine` y emite un token de clase *error*, que el parser puede saltar para continuar y reportar múltiples problemas en una sola pasada. Esta estrategia —continuar tras el error en vez de abortar— mejora notablemente la experiencia frente a un lexer que se detiene en el primer carácter ilegal.

2.2 Parser

El parser se construye a partir de la gramática en `src/parser/hulk.grammar`, escrita con una notación de estilo Bison —directivas `%token`, `%type` y `%start`, y reglas con sus acciones semánticas—. La precedencia de operadores —aritmética, lógica, comparación, concatenación de strings, potencia, negación unaria y operadores de test de tipo— se declara con `%left`, `%right` y `%nonassoc`, que se usan para resolver los conflictos *shift/reduce* de forma determinista.

Esa gramática la procesa un **generador de parsers propio** (`tools/parsergen/`): calcula los conjuntos *FIRST*, construye el autómata canónico de ítems **LR(1)**, fusiona sus estados a **LALR(1)** resolviendo los conflictos con la precedencia declarada, y emite las tablas de acción y goto como código C++. En ejecución, un motor LR dirigido por tablas escrito a mano ejecuta el bucle *shift/reduce* sobre una pila explícita de estados; en cada reducción calcula el *Span* combinado y dispara la acción semántica que construye el nodo del AST. Un `ParserDriver` es dueño del lexer y le entrega los tokens uno a uno mediante un `ParserLexerAdapter` (`yylex`) que traduce el `Token` del proyecto al `Symbol` que esperan las tablas; los errores sintácticos se canalizan por el `DiagnosticEngine`, informando qué tokens se esperaban.

La elección de un esquema LALR(1) —en lugar de un descenso recursivo a mano— responde a que la gramática contiene recursión por la izquierda que LALR maneja de forma natural, a que las declaraciones de precedencia expresan la jerarquía de operadores de modo verificable, y a que la generación de tablas detecta automáticamente las ambigüedades. La decisión adicional de implementar el generador en vez de usar Bison aporta control total sobre las tablas y su integración con los tipos del proyecto (`Token`, `Span`, `Symbol` y el `DiagnosticEngine`), elimina una dependencia externa de *build* y hace explícito y auditable todo el algoritmo de construcción LALR(1).

2.3 Árbol de Sintaxis Abstracta (AST)

El AST está definido en `src/ast/` y contiene aproximadamente cincuenta tipos de nodos agrupados por categoría: literales, operadores binarios y unarios, condicionales, bucles, *bindings*, declaraciones de tipos, de protocolos y de funciones. Hay tres clases base abstractas: `ASTNode`, `Expr` (cualquier cosa que produce un valor) y `Decl` (funciones, tipos, protocolos). Un nodo `Program` contiene una lista de `Decl` más la expresión global final.

Cada clase de nodo concreta implementa `accept(Visitor&)`, habilitando el **patrón Visitor** en todas las fases subsiguientes. Esta es una de las decisiones arquitectónicas más importantes del proyecto: en lugar de dispersar la lógica de cada fase entre los métodos de las clases de nodo (lo que mezclaría responsabilidades de parsing, análisis y generación en una misma clase), cada fase es un visitante independiente. Añadir una nueva fase no requiere tocar las clases de nodo; añadir un nuevo tipo de nodo sí exige extender los visitantes, pero el compilador de C++ lo señala como error de compilación al faltar el `override`, garantizando exhaustividad.

2.4 Análisis semántico

El análisis semántico es orquestado por `SemanticAnalyzer (src/semantic/)` y corre en tres sub-fases encadenadas, cada una productora de un mapa lateral que la siguiente consume (Figura 2).

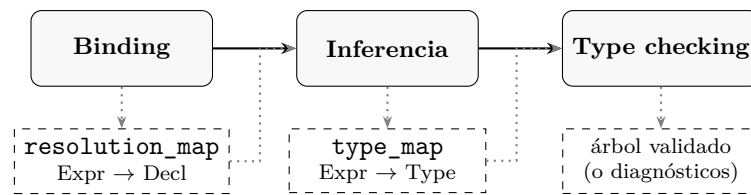


Figura 2: Las tres sub-fases del análisis semántico. Cada una produce un mapa lateral (caja punteada) que las posteriores leen; el AST permanece inmutable.

Binding (src/binding/). Resuelve cada referencia a nombre —variables, llamadas a funciones, nombres de tipos, llamadas a métodos— hacia su declaración. Corre tres pases:

1. El primero registra todas las declaraciones de nivel superior en las tablas semánticas, permitiendo que tipos y funciones mutuamente recursivos funcionen correctamente.
2. El segundo recorre todas las expresiones y guarda la resolución en un `resolution_map` indexado por puntero al nodo AST.
3. El tercero valida ciclos de herencia, compatibilidad de firmas en overrides, aridad de llamadas a funciones y corrección de extensiones de protocolo.

Inferencia de tipos (src/inference/). Determina el tipo de cada expresión mediante iteración de punto fijo. El sistema de tipos tiene siete *kinds*: `Number`, `String`, `Boolean`, `Void`, `Object`, `Unknown` y `Error`. El kind `Error` se propaga: una vez que una sub-expresión es marcada como errónea, sus consumidores también lo son, evitando cascadas de errores. Para expresiones `if`, el tipo inferido es el Ancestro Común Más Bajo (LCA) de las ramas *then* y *else* en la jerarquía de herencia.

Type checking (src/typecheck/). Valida la conformancia $T_1 \preceq T_2$ en todo el árbol anotado: tipos de argumentos en sitios de llamada, tipos de operandos en operadores built-in, compatibilidad de asignación en bindings, y validez de *casts* con *as*.

Las tres sub-fases producen mapas laterales en lugar de anotar los nodos del AST directamente. Esto mantiene el AST como un registro puro de lo que escribió el programador, independiente de cualquier resultado de análisis. La ventaja práctica es doble: el AST puede reusarse por ambos caminos de ejecución sin riesgo de que uno contamine el estado del otro, y los mapas de anotación pueden descartarse o regenerarse sin reconstruir el árbol.

2.5 Evaluador de árbol (Camino A)

`src/eval/` contiene un evaluador basado en el patrón Visitor que ejecuta programas directamente desde el AST anotado. Cada método `visit` computa un `HulkValue` y lo deposita en un slot de resultado. El scoping de variables usa una cadena de `Environment`: cada binding `let` empuja un environment hijo; al salir del binding se hace *pop*.

Este evaluador fue el primer camino de ejecución funcional, utilizado antes de que existiera el backend. Hoy sirve principalmente como oráculo de referencia: para cualquier programa válido, su salida debe coincidir con la de la BannerVM, lo que habilita pruebas diferenciales automáticas.

2.6 Generación de IR (Camino B)

`src/backend/ir_gen.cpp` es un segundo visitor del AST que traduce el árbol anotado a HulkIR, una representación intermedia de nivel medio definida en `src/ir/`. HulkIR es un formato de código de tres direcciones que todavía conoce los tipos, los nombres de campos y la jerarquía de herencia. Los temporales tienen nombres de string (`add_3`, `tmp_call_7`) en lugar de registros numéricos; la asignación de slots ocurre más tarde en la VM.

Decisiones clave de lowering en esta etapa: los bucles `for` se desazucaran a bucles `while` sobre un iterador siguiendo el protocolo `Iterable`; las expresiones `new` emiten una instrucción `NewObject` seguida de una llamada al constructor. Un `NameMangler` produce etiquetas únicas para métodos sobrescritos (`Dog__speak` vs. `Animal__speak`) de modo que cada función en el namespace plano del BannerIR tenga un identificador único.

2.7 Lowering a BannerIR

`src/backend/hulkir_to_banner.cpp` convierte HulkIR a **BannerIR**, un IR de bajo nivel (`src/banner/`) y un formato linealizado con tres secciones:

- **.TYPES:** *layouts* aplanados de clases, incluyendo todos los atributos heredados y una *vtable* que mapea números de *slot* de método a etiquetas de función.
- **.DATA:** *pool* estático de *strings*.
- **.CODE:** funciones de nivel superior con declaraciones explícitas de `PARAM/LOCAL` seguidas de instrucciones de tres direcciones.

El cambio estructural principal respecto a HulkIR es la introducción de instrucciones `PARAM` explícitas antes de cada `CALL` o `VCALL`, respetando la convención de llamada de arquitecturas reales y simplificando el loop de ejecución de la VM.

2.8 BannerVM

La VM (`src/vm/`) compila el `BannerProgram` antes de ejecutarlo. El paso de compilación asigna IDs numéricos a los tipos, distribuye los campos de objetos en *arrays* de *slots* planos—garantizando que los campos heredados ocupen el mismo índice independientemente del tipo dinámico, lo que permite acceso $O(1)$ por índice— y convierte los nombres de temporales a índices numéricos de *frame*.

La ejecución mantiene un `vector<Frame>` como pila de llamadas. Cada *frame* tiene un *program counter*, un *array* de *slots* para locales y temporales, un *buffer* de parámetros para la próxima llamada y un índice de *slot* de retorno. El *loop* principal despacha y avanza el contador o salta a una dirección de etiqueta.

Un recolector de basura mark-and-sweep administra objetos y strings alojados en el heap. Se activa periódicamente según el número de alocaiones, marca todos los objetos alcanzables

desde los frames vivos y libera el resto. Handles generacionales detectan referencias colgantes sin requerir un recorrido completo en cada acceso.

Límites de seguridad configurables mediante `VMOptions` —profundidad máxima de pila, número máximo de instrucciones, tamaño máximo del heap— impiden que recursión infinita o bucles sin fin cuelguen el proceso.

2.9 Invocación del compilador

El proyecto produce varios ejecutables que exponen distintos puntos del pipeline, lo que permite inspeccionar y probar cada etapa por separado. La Tabla 1 resume los binarios y las opciones principales.

Tabla 1: Binarios del proyecto y opciones de línea de comandos.

Binario / opción	Etapla expuesta	Resultado
<code>hulk_semantic</code>	<i>frontend</i> + semántica	diagnósticos
<code>hulk_eval</code>	Camino A (evaluador)	ejecución sobre AST
<code>hulk_backend</code>	Camino B (completo)	ejecutable nativo
<code>--run-banner</code>	BannerVM	ejecuta en la VM
<code>--emit-ir</code>	HulkIR	vuelca <code>.hir</code>
<code>--emit-banner</code>	BannerIR	vuelca <code>.banner</code>
<code>-o <ruta></code>	<i>backend</i>	nombra la salida
<code>--restricted-inference</code>	semántica (todos)	exige anotación/hueco

La existencia de un binario por etapa no es un accidente de empaquetado, sino una consecuencia directa del principio de fases independientes: cada frontera del pipeline es a la vez un punto de inspección y un punto de prueba.

3 Features implementados del lenguaje base

El compilador implementa los features del lenguaje HULK conforme a los apéndices A.2 a A.11 de la especificación oficial.¹

A.2 — Expresiones. El lenguaje es completamente orientado a expresiones: toda construcción (condicionales, bucles, bloques, let-bindings) produce un valor. Se implementan los operadores aritméticos (+, −, *, /, %, ^), concatenación de strings (@, @@), comparación, lógica (&, |, !), asignación destructiva (:=) y los literales. Las funciones y constantes builtin están disponibles: `print`, `sqrt`, `sin`, `cos`, `exp`, `log`, `rand`, `PI`, `E`.

A.3 — Funciones. Se soportan funciones globales con sintaxis inline (`function f(x) =>expr`) y de bloque (`function f(x) { ... }`). Las funciones pueden ser mutuamente recursivas gracias al pase de registro previo al binding.

A.4 — Variables. El binding permite múltiples bindings simultáneos con sombreado de nombres del scope externo. La asignación destructiva `:=` permite mutar una variable ya vinculada dentro de su scope.

A.5 — Condicionales. La construcción `if (cond)` e `elif (cond)` y `else` e está completamente implementada. El tipo del resultado es el LCA de las ramas, garantizando que el condicional siempre produzca un tipo concreto compatible con ambos caminos.

¹Documentación de referencia: <https://matcom.github.io/hulk/appendix-hulk-syntax.html>.

A.6 — Bucles. Se implementan `while (cond) body` y `for (x in iter) body`. El bucle `for` es azúcar sintáctica que el compilador desazucara a un `while` sobre un iterador que sigue el protocolo `Iterable`, conforme a la especificación.

A.7 — Tipos nominales. Las declaraciones `type` crean tipos nominales con herencia simple (`inherits`). Se soportan constructores con parámetros, inicialización de atributos, referencia a la instancia actual con `self`, llamadas al método del padre con `base()`, test de tipo dinámico con `is` y `downcast` explícito con `as`.

A.8 — Type checking. El checker valida la conformancia $T_1 \preceq T_2$ en todo el árbol anotado: argumentos en sitios de llamada, operandos de operadores built-in, compatibilidad en asignaciones y casts. La relación de conformancia sigue la jerarquía de herencia; para protocolos, verifica que el tipo tenga los métodos requeridos con firmas compatibles.

A.9 — Inferencia de tipos. El inferidor de punto fijo determina el tipo de cada expresión. Las anotaciones de tipo son opcionales en todos los lugares del lenguaje; el inferidor las completa cuando se omiten. Si la inferencia falla para algún símbolo, el programa no compila.

A.10 — Protocolos. Las declaraciones `protocol ... extends ...` definen interfaces estructurales. La conformancia es puramente estructural: un tipo satisface un protocolo si provee los métodos requeridos con firmas compatibles, sin ninguna declaración explícita de implementación.

A.11 — Iterables. El protocolo `Iterable` está integrado como builtin del compilador. Los tipos que implementan `next(): Boolean` y `current(): T` son reconocidos como iterables. El bucle `for` se compila a través de este protocolo.

Tabla 2: Resumen de cobertura de la especificación HULK.

Apéndice	Feature	Estado
A.2	Expresiones y operadores	Completo
A.3	Funciones (inline y bloque)	Completo
A.4	Variables y asignación destructiva	Completo
A.5	Condicionales (<code>if/elif/else</code>)	Completo
A.6	Bucles (<code>while, for</code>)	Completo
A.7	Tipos nominales y herencia	Completo
A.8	Type checking / conformancia	Completo
A.9	Inferencia de tipos	Completo
A.10	Protocolos estructurales	Completo
A.11	Iterables	Completo

4 Las representaciones intermedias

El compilador atraviesa tres representaciones distintas entre el AST y la ejecución: `HulkIR`, `BannerIR` y, finalmente, el `BannerProgram` compilado de la VM. Cada transformación baja el nivel de abstracción y acerca el programa a la máquina, eliminando información de alto nivel que ya no es necesaria. Esta sección ilustra el descenso con un ejemplo.

4.1 Ejemplo de descenso

Considérese el siguiente programa fuente:

```
1 function square(n: Number): Number => n * n;
2 print(square(7));
```

Listing 1: Programa fuente de ejemplo.

HulkIR. El primer IR conserva los tipos y usa temporales con nombre. La multiplicación y la llamada se desazucaran en código de tres direcciones:

```
1 func square(n : Number) -> Number {
2   mul_1 = n * n           ; Number
3   return mul_1
4 }
5 entry {
6   call_1 = call square(7) ; Number
7   call_2 = call print(call_1)
8 }
```

Listing 2: HulkIR (esquemático). Temporales con nombre, tipos aún presentes.

BannerIR. El lowering a BannerIR introduce instrucciones `PARAM` explícitas antes de cada llamada y separa las secciones `.CODE/.DATA/.TYPES`:

```
1 .CODE
2 square:
3   PARAM n
4   mul_1 = MUL n, n
5   RET mul_1
6 entry:
7   PARAM 7
8   call_1 = CALL square
9   PARAM call_1
10  call_2 = CALL print
```

Listing 3: BannerIR (esquemático). Aparecen los `PARAM` explícitos, propios de una convención de llamada.

BannerProgram compilado. Finalmente, la VM compila el BannerIR: los temporales con nombre (`mul_1`, `call_1`) se convierten en índices numéricos de slot dentro del frame, los tipos reciben IDs numéricos y los campos de objetos se distribuyen en arrays planos. Tras esta compilación ya no quedan nombres simbólicos, lo que habilita despacho y acceso a slots.

4.2 Por qué tres niveles

La separación responde a una división clara de responsabilidades: HulkIR es el nivel donde el conocimiento de tipos y herencia todavía es útil; BannerIR es el nivel donde se materializa la convención de llamada y la linealización; y el BannerProgram es el nivel donde todo es numérico y listo para un loop de despacho rápido. Cada frontera entre niveles es un punto natural de prueba: se puede emitir y comparar cada IR de forma independiente (`--emit-ir`, `--emit-banner`, `--emit-banner-compiled`).

5 El sistema de tipos en detalle

Antes de abordar la extensión conviene precisar el sistema de tipos sobre el que opera, pues sus propiedades condicionan tanto el algoritmo de inferencia como el alcance de los *type holes*.

5.1 Kinds de tipo

El inferidor (`src/inference/hulk_type.h`) representa cada tipo mediante un kind y, cuando aplica, un nombre nominal.

Tabla 3: Kinds del sistema de tipos y su rol.

Kind	Habitantes	Rol
Number	literales numéricos, aritmética	tipo base
String	literales de cadena, @	tipo base
Boolean	true/false, lógica	tipo base
Void	efectos sin valor útil	resultado de sentencias
Object	raíz de la jerarquía nominal	supertipo universal
Unknown	hueco aún no resuelto	estado transitorio
Error	expresión mal tipada	absorbente

Los tipos nominales declarados con `type` se modelan como refinamientos de `Object` con un nombre y una posición en la jerarquía de herencia. `Unknown` y `Error` son kinds internos, nunca escribibles por el programador: el primero es el valor inicial de todo hueco; el segundo es un elemento absorbente que detiene la propagación de diagnósticos.

La Figura 3 ilustra la jerarquía: `Object` es la raíz universal, bajo la cual cuelgan los tipos base y la jerarquía nominal definida; `Error` y `Unknown` viven fuera de ella como estados especiales del proceso de inferencia.

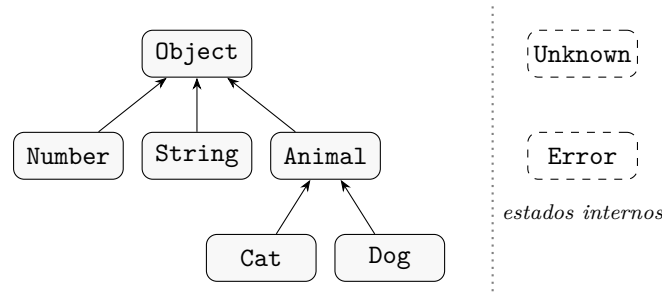


Figura 3: Jerarquía de tipos. Las cajas sólidas forman la red de conformancia (con `Object` como supertipo universal); las cajas punteadas son los kinds internos `Unknown` (hueco no resuelto) y `Error` (absorbente), ajenos a la jerarquía visible.

5.2 La relación de conformancia

La regla (**Protocolo**) captura el subtipado estructural: la pertenencia no se declara, se *verifica* comprobando que el tipo provea cada método requerido por el protocolo con una firma compatible (parámetros contravariantes y retorno covariante, en el caso ideal; el compilador implementa la comprobación de firma exacta de nombre y aridad).

5.3 El Ancestro Común Más Bajo (LCA)

Para una expresión condicional `if (c) e1 else e2`, el tipo resultante no puede ser simplemente el de una rama: debe ser un supertipo de ambas. El inferidor calcula el **Ancestro Común Más Bajo** (*Lowest Common Ancestor*, LCA) de los tipos de las dos ramas en la jerarquía de herencia. Informalmente, $LCA(T_1, T_2)$ es el tipo más específico U tal que $T_1 \preceq U$ y $T_2 \preceq U$.

```

1 type Animal { }
2 type Dog inherits Animal { }
3 type Cat inherits Animal { }
4
5 let pet = if (rand() > 0.5) new Dog() else new Cat() in pet;
6 // tipo inferido de pet: Animal (= LCA(Dog, Cat))

```

Listing 4: Ejemplo de LCA. El if produce `Animal`, supertipo común de `Dog` y `Cat`.

Cuando las ramas no comparten ancestro común más allá de `Object`, el resultado es `Object`; cuando una rama es `Error`, el LCA es la otra rama (consecuencia de las reglas de absorción), de nuevo para no propagar ruido. El cálculo de LCA es la pieza que permite que el condicional —una de las expresiones más frecuentes— siempre produzca un tipo concreto compatible con ambos caminos sin exigir anotación del programador.

6 Extensión a HULK: type holes explícitos

Más allá de la especificación base, el compilador implementa **type holes explícitos**: una extensión sintáctica y semántica que permite al usuario marcar posiciones específicas de anotación con `_` o `auto` para solicitar inferencia de tipos *en ese punto concreto*. Esta sección define la extensión formalmente, analiza su semántica, describe su algoritmo de resolución, y discute la motivación de diseño.

6.1 Motivación

El lenguaje base de HULK ofrece dos extremos para la anotación de tipos: omitir por completo la anotación (inferencia total) o escribir el tipo concreto (anotación total). Esta dicotomía esconde una ambigüedad pragmática: cuando un parámetro aparece *sin* anotación, no hay forma de distinguir si el usuario decidió deliberadamente delegar el tipo al compilador o si simplemente olvidó anotarlo. En equipos y en contextos didácticos, esa diferencia importa: una omisión involuntaria es un defecto a corregir, mientras que una delegación deliberada es una decisión de diseño legítima.

Los *type holes* resuelven esta ambigüedad introduciendo un *tercer estado* explícito. El programador puede ahora expresar tres intenciones distintas en cada posición de anotación, como muestra la Tabla 4.

Tabla 4: Los tres estados de anotación habilitados por la extensión.

Forma sintáctica	Intención	Bajo <code>--restricted</code>
<code>x: Number</code>	“El tipo es <code>Number</code> ”	Válida
<code>x: _ / x: auto</code>	“Infiere el tipo aquí”	Válida
<code>x</code> (sin anotación)	(ambigua)	Error

6.2 Sintaxis

La gramática extiende la producción de tipos (`type_expr` en `src/parser/grammar.y`) con dos nuevas alternativas terminales, `UNDERSCORE` y `AUTO`:

Ambas formas son semánticamente idénticas; `_` es más compacto y se lee como un hueco, mientras que `auto` es más descriptivo y resulta familiar para quien conoce C++. Son válidas en todas las posiciones de anotación: tipos de bindings `let`, tipos de parámetros de función, tipos de retorno, parámetros y retornos de métodos, y anotaciones de tipo de atributos.

```

1 // hueco en un binding let
2 let x: auto = 42 in print(x);
3
4 // hueco en parametro y retorno de funcion
5 function square(n: _): _ => n * n;
6
7 // equivalencia de _ y auto en la misma firma
8 function add(a: auto, b: _): auto => a + b;

```

Listing 5: Ejemplos de uso de type holes en distintas posiciones de anotación.

6.3 Semántica

Cada `_` o `auto` introduce un *hueco de tipo*: un nodo `InferTypeAnnotation` en el AST. Durante la fase de inferencia, los huecos se tratan como variables de tipo frescas. El inferidor recolecta restricciones del entorno de la expresión y resuelve cada hueco al único tipo concreto consistente con todas sus restricciones. Algunos ejemplos de recolección de restricciones:

- `x + 1` restringe el hueco de `x` a `Number`.
- `!x` restringe el hueco de `x` a `Boolean`.
- `x * x` restringe tanto el parámetro como el tipo de retorno a `Number`.
- `x @ "s"` restringe el hueco de `x` a un tipo concatenable (`String` o `Number`, según la semántica de `@`).

Si un hueco tiene una única resolución válida, se sustituye y el *type checker* recibe un árbol completamente anotado, equivalente a uno donde el programador hubiera escrito el tipo inferido explícitamente. Si un hueco *no* tiene resolución posible (falta de información) o si dos restricciones sobre el mismo hueco son incompatibles (por ejemplo, una variable usada en contexto aritmético y booleano a la vez), el compilador emite un error descriptivo con las restricciones en conflicto y una sugerencia de anotación explícita.

6.4 Algoritmo de resolución

La resolución de huecos se integra en el inferidor de punto fijo existente, sin requerir un solucionador de restricciones separado. El procedimiento se resume en el Algoritmo 1.

Algoritmo 1 (Resolución de type holes por punto fijo).

1. Inicializar el tipo de cada hueco a `Unknown`.
2. **Repetir** hasta convergencia o hasta `max_iterations`:
 - a) Recorrer el AST anotando el tipo de cada expresión en función de los tipos actuales de sus sub-expresiones.
 - b) Cuando una expresión impone una restricción sobre un hueco aún en `Unknown`, refinar el hueco al tipo impuesto y marcar `changed = true`.
 - c) Si un refinamiento entra en conflicto con un valor concreto previo del hueco, marcar el hueco como `Error` y registrar un diagnóstico.
3. Tras la convergencia, todo hueco que permanezca en `Unknown` carece de información suficiente: registrar un error “no se pudo inferir”.
4. Sustituir cada hueco resuelto por su tipo concreto y entregar el árbol anotado al type checker.

La elección de reutilizar el inferidor de punto fijo —en lugar de un algoritmo de unificación estilo Hindley-Milner con union-find— es deliberada y se justifica en la Sección 7. El sistema de tipos de HULK no es paramétricamente polimórfico: no hay variables de tipo de primer orden ni generics. En ese contexto, la inferencia se reduce a propagar tipos concretos a través de un grafo de restricciones monomórfico, tarea para la cual el punto fijo iterativo es suficiente, predecible y fácil de razonar. La terminación está garantizada porque la red de tipos es finita (siete kinds más la jerarquía nominal) y cada refinamiento es monótono: un hueco solo pasa de `Unknown` a un tipo concreto o a `Error`, nunca de vuelta. La Figura 4 representa esta monotonía como una máquina de estados: los estados son absorbentes una vez que el hueco abandona `Unknown`, lo que garantiza que el punto fijo no oscile.

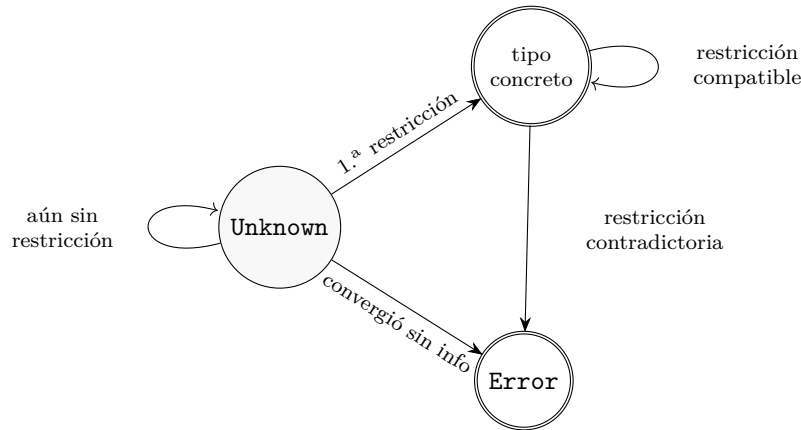


Figura 4: Ciclo de vida de un *type hole*. Los estados *tipo concreto* y *Error* son absorbentes (doble círculo): una vez alcanzados, el hueco no vuelve a `Unknown`, garantizando la monotonía y la terminación del punto fijo.

6.5 Diagramas de inferencia paso a paso

Para hacer concreto el funcionamiento del algoritmo, se presentan dos diagramas completos de resolución de huecos.

Caso 1: parámetro y retorno inferidos. Considérese la función:

```
1 function square(n: _): _ => n * n;
```

El AST asocia dos huecos: h_n (tipo de n) y h_r (tipo de retorno). La Tabla 5 muestra la evolución del punto fijo.

Tabla 5: Traza de inferencia de `square`.

Iter.	h_n	h_r	Restricción aplicada
0	Unknown	Unknown	(inicial)
1	Number	Unknown	$n * n$ exige $n : \text{Number}$
2	Number	Number	cuerpo <code>Number</code> fija el retorno
3	Number	Number	sin cambios $\Rightarrow$ convergió

Tras la convergencia, el árbol entregado al type checker es idéntico al de `function square(n: Number): Number => n * n`.

Caso 2: hueco sin información suficiente. Considérese:

```
1 function id(x: _): _ => x;
```

Aquí x se usa solo como valor de retorno sin imponer ninguna restricción de tipo. El punto fijo converge con h_x y h_r ambos ligados entre sí pero *sin* un tipo concreto: permanecen en `Unknown`. El algoritmo, en su paso final, detecta el `Unknown` residual y emite un error de “no se pudo inferir”, con la sugerencia de anotar explícitamente. Este caso ilustra una limitación inherente a la inferencia monomórfica sin polimorfismo: la función identidad, perfectamente expresable como `forall a. a -> a` en un sistema HM, no tiene un tipo monomórfico único y por tanto no puede inferirse sin anotación.

6.6 Casos de error de la extensión

La extensión distingue con precisión tres situaciones de fallo, cada una con un diagnóstico propio:

1. **Restricciones contradictorias.** El mismo hueco recibe dos tipos concretos incompatibles:

```
1 function bad(x: _): _ => if (x) x + 1 else 0;
2 // x usado como Boolean (condición) y como Number (x + 1)
3 // -> error: restricciones en conflicto sobre el hueco de x
```

2. **Información insuficiente.** El hueco no recibe ninguna restricción concreta (caso 2 anterior): error “no se pudo inferir” con sugerencia de anotación.
3. **Omisión bajo modo restringido.** Con `--restricted-inference`, una posición sin anotación ni hueco produce el error de política descrito en la siguiente subsección.

La separación de estos tres casos es una ventaja diagnóstica frente a un sistema que tratara toda falla de inferencia con un único mensaje genérico: el usuario sabe de inmediato si debe *desambiguar* (caso 1), *añadir contexto* (caso 2) o *declarar su intención* (caso 3).

6.7 La bandera `--restricted-inference`

Cuando la bandera `--restricted-inference` está activa, omitir una anotación de tipo es un error de compilación; solo `_`, `auto` o un nombre de tipo explícito son aceptables. Esto hace ejecutable en tiempo de compilación la distinción entre “el usuario olvidó anotar” y “el usuario pidió deliberadamente inferencia”.

La verificación la realiza un visitante dedicado, que recorre declaraciones y expresiones comprobando cada posición de anotación. El núcleo de la política es el predicado `require_annotation`:

```
1 void require_annotation(bool has_concrete_annotation,
2                         bool is_type_hole,
3                         const Span& span) {
4     if (restricted_inference_
5         && !has_concrete_annotation
6         && !is_type_hole)
7         report_restricted(span);
8 }
```

Listing 6: Núcleo del chequeo de política (`analyzer.cpp`). El error solo se emite cuando la posición no tiene ni anotación concreta ni hueco explícito.

El mensaje emitido es explícito y orientado a la acción:

“Inferencia implícita no permitida en modo restringido. Use ‘: _’, ‘: auto’ o escriba un tipo concreto.”

Es importante notar que la bandera *no* cambia el algoritmo de inferencia: un programa con huecos `_`/auto se compila de forma idéntica con y sin la bandera. Lo único que cambia es que, con la bandera, la omisión total de la anotación deja de ser aceptable. La extensión y la bandera son por tanto ortogonales: la primera añade una capacidad expresiva, la segunda añade una política que la aprovecha.

6.8 Cobertura de pruebas de la extensión

La extensión cuenta con pruebas dedicadas. Cubren: equivalencia entre `_` y `auto`, huecos en bindings, parámetros, retornos y atributos, inferencia que fuerza un tipo `Boolean` o `Number`, múltiples parámetros con huecos, y la interacción con el modo restringido. El backend ejecuta cada caso tanto en modo normal como con `--restricted-inference`, garantizando que la presencia de la bandera no altera la semántica de un programa correctamente anotado.

7 Análisis comparativo con otros lenguajes

Esta sección sitúa la extensión de *type holes* y las decisiones de diseño del compilador en el contexto de lenguajes contemporáneos, justificando por qué las elecciones tomadas son razonables para los objetivos de HULK.

7.1 El placeholder de tipo a través de los lenguajes

La idea de un *marcador de posición* que delega el tipo al compilador no es nueva; existe en numerosos lenguajes, aunque con semánticas y alcances distintos. La Tabla 6 resume las variantes más conocidas.

Tabla 6: Placeholders de inferencia local en lenguajes representativos.

Lenguaje	Sintaxis	Alcance del placeholder
C++ (C++11+)	<code>auto</code>	Variables locales, retorno (C++14)
Rust	<code>let x = ...; _</code>	Locales; <code>_</code> en posiciones de tipo parcial
C#	<code>var</code>	Variables locales
Go	<code>x := ...</code>	Variables locales
Scala	inferencia + <code>_</code>	Locales; <code>_</code> como comodín de tipo
Kotlin	<code>val x = ...</code>	Variables locales y de propiedad
TypeScript	inferencia contextual	Locales, retorno, contextual
Haskell	<code>_</code> (typed hole)	Cualquier posición; reporta el tipo esperado
HULK	<code>_</code> / <code>auto</code>	Cualquier posición de anotación

Dos observaciones merecen comentario.

Generalidad posicional. La mayoría de lenguajes como C++, C#, Go, Kotlin limitan su placeholder a *variables locales*: `auto`/`var`/`:=` no pueden usarse en la firma de una función para inferir el tipo de un parámetro. La extensión de HULK es más general en ese eje: un hueco es válido en *cualquier* posición de anotación, incluidos parámetros, retornos y atributos. Esto la acerca más a la inferencia posicionalmente libre de TypeScript o Scala que al `auto` restringido de C++. La generalidad es viable precisamente porque el inferidor de HULK ya opera global y bidireccionalmente sobre el cuerpo de las funciones; el hueco no es más que un punto de anclaje al que el inferidor adjunta una variable de tipo.

Doble sintaxis `_`/auto. Pocos lenguajes ofrecen *dos* grafías intercambiables para el mismo concepto. La decisión aquí es ergonómica: `_` es minimalista y reconocible como “hueco” por quien viene de Rust o Haskell, mientras que `auto` es autoexplicativo y cómodo para quien viene de C++. Al ser estrictamente equivalentes, no introducen carga cognitiva adicional: no hay una regla sutil que distinga cuándo usar una u otra.

Comparación concreta de código. La generalidad posicional se aprecia mejor con un ejemplo paralelo. Supóngase que se quiere inferir el tipo de un parámetro a partir de su uso. En C++, C# o Go esto *no* es posible con su placeholder local:

```

1 // C++: 'auto' en parametro requiere C++20 (abbreviated templates),
2 //     y aun asi crea un template, no una funcion monomorfica.
3 auto square(auto n) { return n * n; } // genera un template
4
5 // C#: 'var' NO es valido en parametros
6 //     int Square(var n) => n * n; // error de compilacion
7
8 // Go: no existe placeholder para el tipo de un parametro
9 //     func square(n) int { return n*n } // error de sintaxis
10
11 // HULK: el hueco en el parametro se infiere a Number
12 function square(n: _): _ => n * n; // OK -> (Number): Number

```

Listing 7: El placeholder local de C++/C#/Go no alcanza la firma. Solo HULK infiere el parámetro.

La diferencia de fondo es que `auto`/`var` en esos lenguajes son *inferencia de tipo de variable a partir de su inicializador* —una inferencia *local y unidireccional*— mientras que el hueco de HULK es un punto de anclaje para una inferencia *global sobre el cuerpo* de la función. Esta es la misma capacidad que ofrece la inferencia de TypeScript para retornos, o Scala para gran parte de las firmas, pero HULK la unifica bajo una única sintaxis explícita y la hace disponible en *toda* posición de anotación.

7.2 Typed holes: la inspiración de Haskell e Idris

El término *type hole* (hueco de tipo) proviene de Haskell (extensión *typed holes*, GHC) y de lenguajes con tipos dependientes como Idris y Agda. En esos sistemas, un `_` en una expresión provoca que el compilador *reporte* el tipo que esa posición debería tener, sirviendo como una herramienta de desarrollo interactivo: el programador escribe `_`, el compilador responde “aquí se espera un valor de tipo `Int -> Bool`”, y el usuario completa el hueco.

La extensión de HULK comparte el *nombre* y el espíritu —un `_` que el sistema de tipos debe “rellenar”— pero difiere en el rol: en HULK el hueco es de inferencia productiva (el compilador deduce el tipo y continúa la compilación), no de interrogación (el compilador solo informa y exige al humano completar). Esta diferencia es coherente con la naturaleza de HULK: un sistema de tipos sin polimorfismo paramétrico donde, para una posición dada, casi siempre existe un *único* tipo concreto consistente, por lo que reportarlo y exigir al humano que lo transcriba sería trabajo redundante. La elección de *inferir y seguir* en lugar de *interrogar y detener* es, por tanto, la adecuada para el dominio.

7.3 Por qué punto fijo y no Hindley-Milner

Una pregunta natural es por qué el compilador usa inferencia por iteración de punto fijo en lugar del algoritmo de Hindley-Milner (HM), estándar en la familia ML (OCaml, Haskell, Standard ML). La respuesta está en la naturaleza del sistema de tipos de HULK:

- **Sin polimorfismo paramétrico.** HM brilla cuando hay variables de tipo cuantificadas universalmente (`forall a. a -> a`) que deben unificarse e instanciarse. HULK no tiene *generics*: cada expresión tiene un tipo concreto monomórfico. Sin variables de tipo de primer orden, la maquinaria de unificación y generalización de HM es innecesaria.
- **Subtipado nominal.** HULK tiene herencia y, por tanto, subtipado (`Dog ≤ Animal`). HM clásico *no* maneja subtipado; extenderlo a subtipado requiere inferencia con restricciones de subtipo (*subtype constraints*), que es considerablemente más compleja. El enfoque de punto fijo con LCA para condicionales modela el subtipado de forma directa y comprensible.
- **Predecibilidad y diagnósticos.** La iteración de punto fijo produce un estado intermedio inspeccionable en cada paso, lo que facilita generar mensajes de error que señalan *cuál* restricción falló. La unificación de HM, en cambio, es conocida por producir errores de tipo difíciles de interpretar (como “cannot unify a with b” en una posición alejada de la causa real).
- **Terminación trivial.** Con una red de tipos finita y refinamientos monótonos, la convergencia está garantizada en pocas iteraciones.

En resumen, HM sería sobreingeniería para un sistema de tipos sin polimorfismo paramétrico y con subtipado nominal. El punto fijo iterativo es la herramienta correcta para este dominio, del mismo modo que los análisis de flujo de datos clásicos son la herramienta correcta para el análisis de programas.

7.4 Conformancia estructural vs. nominal

HULK combina dos disciplinas de subtipado que la mayoría de lenguajes mantienen separadas:

- **Subtipado nominal** para tipos `type ... inherits ...`, como en Java, C++ o C#: un tipo es subtipo de otro solo si lo declara explícitamente.
- **Subtipado estructural** para protocolos, como en Go (interfaces implícitas), TypeScript (tipado estructural): un tipo satisface un protocolo si *tiene la forma adecuada*, sin declaración explícita de implementación.

La decisión de borrar los protocolos antes del backend (*type erasure*) sigue el modelo de Go y de los generics de Java: los protocolos son un instrumento de verificación en tiempo de compilación y no necesitan representación en tiempo de ejecución, lo que simplifica el IR y la VM.

7.5 Mapas laterales vs. anotación en el nodo

La decisión de almacenar los resultados de análisis en mapas laterales (`unordered_map<Expr*, x>`) en lugar de campos mutables en los nodos del AST contrasta con la práctica habitual en muchos compiladores didácticos, donde cada nodo lleva un campo `inferredType` que las fases rellenan. El enfoque de HULK se asemeja más al de compiladores que mantienen el árbol de parsing inmutable y construyen *tablas de símbolos* y *mapas de tipos* aparte, como hace el compilador de Roslyn (C#) con sus *bound nodes* separados del *syntax tree*. La ventaja, ya mencionada, es que el AST permanece como un registro fiel del texto fuente y ambos caminos de ejecución pueden compartirlo sin interferencia.

7.6 Modelo de ejecución: VM de pila frente a compilación nativa

La elección de ejecutar sobre una VM de pila propia —en lugar de generar código nativo (vía LLVM, por ejemplo) o transpilar a C— se alinea con el modelo de Python (CPython), Java (JVM) y Lua. Una VM de pila ofrece tres ventajas decisivas sobre la compilación nativa:

- **Control total del entorno.** La VM puede imponer límites de pila, instrucciones y heap (`VMOptions`), conteniendo recursión infinita y bucles sin fin.
- **Gestión de memoria simple.** Implementar un *mark-and-sweep* sobre el heap de la VM es directo; integrar un GC en código nativo es un proyecto en sí mismo. La VM tiene visibilidad completa de la raíz del grafo de objetos (los frames vivos), lo que el marcado aprovecha.
- **Portabilidad y depuración.** El `BannerProgram` es independiente de la arquitectura; el mismo programa corre en cualquier plataforma sin recompilar el backend. Además, el loop de despacho es inspeccionable instrucción a instrucción, facilitando la depuración del propio compilador.

El costo es el rendimiento: una VM de pila interpretada es órdenes de magnitud más lenta que código nativo. Para los objetivos de HULK —corrección, claridad y seguridad por encima de velocidad— el intercambio es claramente favorable, la misma decisión que tomaron los diseñadores de CPython y de la JVM original.

8 Manejo de errores y diagnósticos

El compilador HULK centraliza todo el reporte de errores en un `DiagnosticEngine` compartido por todas las fases.

8.1 Spans y localización

Cada token, y por extensión cada nodo del AST, lleva un `Span`: un par de posiciones (línea, columna) de inicio y fin en el archivo fuente. El span se propaga sin pérdida desde el lexer hasta las fases semánticas, de modo que un error detectado durante la inferencia o el type checking puede señalar la posición exacta del código culpable, no la de un nodo aproximado. Esta trazabilidad posicional es la base de mensajes accionables como el de `--restricted-inference`, que apunta a la posición concreta de la anotación omitida.

8.2 Niveles y severidades

Los diagnósticos se clasifican por nivel (según la fase que los emite: léxico, sintáctico, semántico) y por severidad (`Error`, `Warning`). El motor acumula todos los diagnósticos de una corrida en lugar de abortar en el primero, lo que permite reportar múltiples problemas en una sola compilación. Esta acumulación es coherente con la recuperación de errores del lexer y del parser descrita en la Sección 2: las fases tempranas producen tokens y nodos de clase `error` en lugar de detenerse, y las fases posteriores los toleran propagando el kind `Error`.

8.3 La propagación absorbente de Error

El uso de `Error` como elemento absorbente del sistema de tipos es una técnica clave de control de ruido. Sin ella, un único error de tipo en una sub-expresión profunda generaría un diagnóstico en cada expresión que la contiene, inundando la salida con mensajes derivados. Al absorber, el compilador reporta el error *una vez*, en su origen, y silencia a sus consumidores.

9 Estrategia de pruebas

La arquitectura del compilador está deliberadamente diseñada para ser testeable por fases, y el conjunto de pruebas lo explota en varios niveles.

9.1 Pruebas de salida esperada por fase

Para cada fase del frontend existe un directorio de salidas esperadas —`tests/expected/semantic/`, `tests/expected/typecheck/`, `tests/expected/eval/` y `tests/expected/backend/`. El harness ejecuta el compilador sobre cada entrada, captura la salida y la compara contra el archivo `.expected` correspondiente. Como cada IR puede emitirse de forma independiente (`--emit-ir`, `--emit-banner`, etc.), una regresión queda localizada en la etapa exacta donde la salida diverge.

9.2 Pruebas diferenciales

El evaluador de árbol (Camino A) y la BannerVM (Camino B) constituyen dos implementaciones independientes de la misma semántica. Para cualquier programa válido, su salida debe coincidir. Esta redundancia se explota como *prueba diferencial*: una divergencia entre ambos caminos delata un error en uno de los dos sin necesidad de un oráculo externo. Es la misma técnica que emplean proyectos como los *fuzzers* de compiladores (CSmith) y los bancos de prueba de motores de JavaScript.

9.3 Pruebas end-to-end y de la extensión

Los casos `tests/end-to-end/` compilan y ejecutan programas completos, verificando el comportamiento observable. La extensión de *type holes* tiene cobertura dedicada (Sección 6): los casos de `08_inference/` validan la inferencia de huecos en *bindings*, parámetros, retornos y atributos; los de `09_restricted/` validan la política de la bandera. El backend ejecuta cada caso en ambos modos, confirmando la ortogonalidad entre la extensión y la bandera.

10 Limitaciones notables

GC síncrono. El recolector *mark-and-sweep* corre sincrónicamente durante la alocaación. Para programas con heaps grandes o alocaación frecuente, esto puede introducir pausas observables. Un recolector incremental o generacional mitigaría esto, pero está fuera del alcance de la implementación actual.

Mensajes de error sin recuperación cross-fase. Si el binding encuentra errores, las fases de inferencia y type checking continúan sobre el árbol parcialmente resuelto, pero los diagnósticos producidos en esas fases posteriores pueden referirse a nodos cuya resolución fue marcada como `Unresolved`. Esto genera en ocasiones errores redundantes o mensajes confusos que apuntan a consecuencias del error original en lugar de a su causa raíz. Un sistema de supresión de errores derivados —que silencie automáticamente los diagnósticos cuyo origen sea un nodo ya reportado— reduciría el ruido en la salida de error.

Inferencia monomórfica. Al carecer de polimorfismo paramétrico, la inferencia no puede deducir tipos genéricos; cada hueco debe resolverse a un único tipo concreto. Esto es coherente con el diseño del lenguaje, pero limita la reutilización de código frente a lenguajes con generics.

11 Conclusiones

El compilador HULK implementa un pipeline completo desde el texto fuente hasta la ejecución en una máquina virtual, cubriendo análisis léxico, parsing LALR(1), análisis semántico en tres fases (binding, inferencia y type checking), generación de IR, lowering y una VM de pila con recolector de basura. Los principios de diseño centrales —patrón Visitor para el recorrido del AST, nodos inmutables con mapas laterales de anotación— dan como resultado una base de código donde cada capa tiene una responsabilidad única y clara, y cada fase puede probarse de forma independiente.

La extensión propuesta —*type holes* explícitos con `_` y `auto`, complementada por la bandera `--restricted-inference`— extiende el lenguaje con un tercer estado de anotación que desambigua la intención del usuario entre “infiere aquí” y “olvidé anotar”. El análisis comparativo muestra que la extensión es *posicionalmente más general* que el `auto/var` de la mayoría de lenguajes mainstream, hereda el espíritu de los *typed holes* de Haskell adaptándolo a un rol de inferencia productiva, y se apoya en un algoritmo de punto fijo que es la elección adecuada para un sistema de tipos sin polimorfismo paramétrico y con subtipado nominal. Junto con la bandera, ofrece un espectro de requerimientos de anotación aplicables a cualquier programa HULK.